

# Regularización y estrategias de modelamiento

Qué penalizar, qué ajustar, qué transformar y cuándo combinar modelos.

---

**Sofia Dextre**

Hack with DSC | PUCP

- 1 L1, L2 y Elastic Net.
- 2 Regularización según el algoritmo.
- 3 Segmentación, ensambles e híbridos.

## 01

# L1, L2 y Elastic Net

---

Cómo elegir la penalización según las características de tus variables.

# Regularizar es ponerle precio a la complejidad

El modelo sigue buscando minimizar el error, pero algunas soluciones ahora tienen un costo.

$$\text{Objetivo} = \text{Loss} + \lambda \cdot \text{Penalización}(\beta)$$

## LOSS

### Ajuste a los datos

¿Qué tan bien explica  
el modelo lo observado?

 $\lambda$ 

### Intensidad

¿Cuánto quiero  
regularizar?

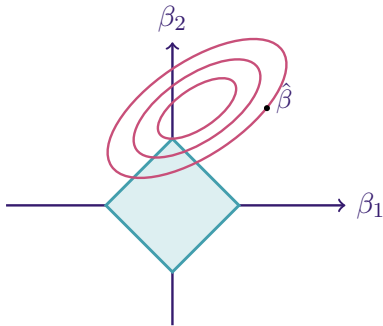
## PENALIZACIÓN

### Estructura

¿Qué tipo de solución  
quiero castigar?

# Regularización Lasso

**Penalización L1:** favorece soluciones sparse y puede llevar coeficientes a cero.



Las esquinas de la región L1 favorecen soluciones con  $\beta_j = 0$ .

## PENALIZACIÓN L1

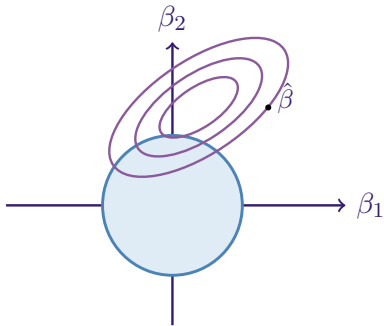
$$\text{Loss} + \lambda \sum_{j=1}^p |\beta_j|$$

### ¿Qué provoca?

- 01 Coeficientes exactamente en 0
- 02 Selección implícita de variables

# Regularización Ridge

**Penalización L2:** reduce los coeficientes de forma suave y favorece soluciones más estables.



La frontera suave de L2 favorece soluciones con coeficientes reducidos, pero normalmente distintos de cero.

## PENALIZACIÓN L2

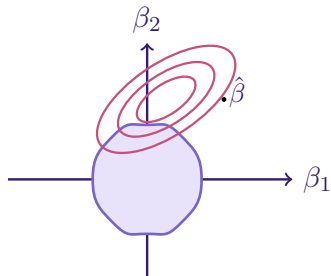
$$\text{Loss} + \lambda \sum_{j=1}^p \beta_j^2$$

### ¿Qué provoca?

- 01 Coeficientes más pequeños
- 02 Mayor estabilidad ante variables correlacionadas

# Regularización Elastic Net

Penalización mixta: combina L1 y L2.



Combina sparsity de L1 con estabilidad de L2.

## MEZCLA L1 + L2

$$\text{Loss} + \lambda \left[ \alpha \sum |\beta_j| + (1 - \alpha) \sum \beta_j^2 \right]$$

$\alpha$  Mezcla entre L1 y L2

$\lambda$  Intensidad de regularización

**Selección + estabilidad**

# L1 y L2 en Credit Scoring: ¿qué aportan?

## L1 · Parsimonia

$$\beta_j \rightarrow 0$$

Menos variables activas

## L2 · Estabilidad

$$|\beta_j| \downarrow$$

Coeficientes más controlados

## Explicabilidad

Dirección y contribución

Coeficientes · SHAP

## Trazabilidad y monitoreo

Variables → Score → Probabilidad

Reconstruir y revisar el modelo

## Hora de practicar

Lasso • Ridge • Elastic Net

[Abrir notebook en Google Colab](#)

# 02

## Regularización implícita

---

Reducir el sobreajuste sin añadir una penalización al modelo.

# Regularización implícita

Reducir el sobreajuste sin añadir una penalización al modelo.

### Regularización explícita

$\text{Loss} + \lambda \cdot \text{Penalización}$

Modificamos la función objetivo

**L1 · L2 · Elastic Net**

### Regularización implícita

Loss

Cambiamos cómo aprende el modelo

**Entrenamiento · Stop-  
ping · Aleatoriedad**

## Early Stopping: saber cuándo detener el entrenamiento

Se usa en modelos que aprenden iterativamente: **Boosting**, redes neuronales y otros algoritmos iterativos.

### Ejemplo: Boosting

Iteración 1 + Árbol 1

Iteración 2 + Árbol 2

Iteración 3 + Árbol 3

⋮

Iteración  $m$  + Árbol  $m$

Learning rate: cuánto aporta cada nuevo árbol.

### ¿Cuándo detenernos?



**Early Stopping:** detener cuando validación deja de mejorar.

## Subsample y Colsample: no todos los árboles ven lo mismo

Introducir diversidad durante el entrenamiento puede ayudar a reducir el sobreajuste.

### Subsample = filas

10 000 clientes

subsample = 0.8



Árbol 1 → 8 000 clientes

Árbol 2 → otros 8 000

Árbol 3 → otros 8 000

Cada árbol recibe una muestra distinta de observaciones.

### Colsample = variables

100 variables

colsample = 0.7



Árbol 1 → 70 variables

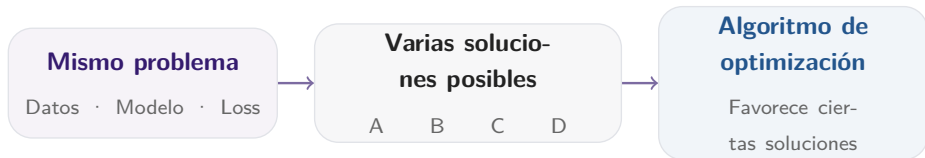
Árbol 2 → otras 70

Árbol 3 → otras 70

Cada árbol recibe un subconjunto distinto de variables.

## El entrenamiento también elige

Varias soluciones pueden ajustar igual de bien al train, pero no generalizar igual.



|            | Train | Validación  |
|------------|-------|-------------|
| Solución A | 0.95  | 0.78        |
| Solución B | 0.95  | <b>0.86</b> |

No dejes esa preferencia a ciegas: **valida fuera del train.**

# Hora de practicar

Early Stopping • Subsample • Colsample

**Abrir notebook en Google Colab**

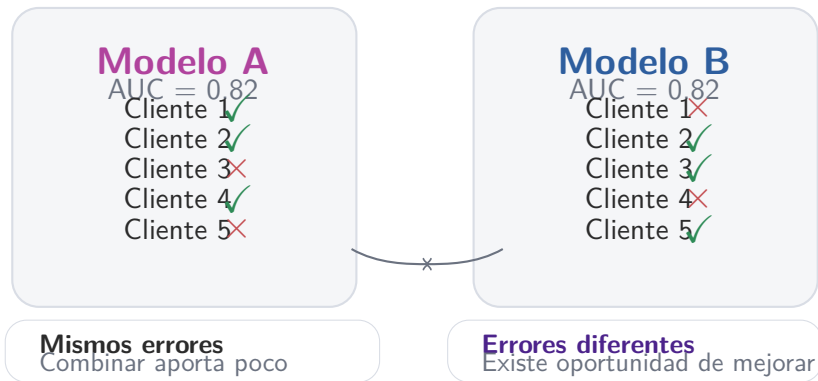
# 03

## Ensembles, Stacking y Modelos Híbridos

---

Cuándo combinar modelos en lugar de seguir ajustando uno solo.

## ¿Por qué combinar modelos?



## Ensembles: varios modelos, una sola predicción

Un **ensemble** combina varias predicciones en una única salida.



El valor aparece cuando los modelos capturan **información complementaria**.

## Ensembles: ¿cuánto pesa cada modelo?

Los modelos pueden aportar distinto. **Los pesos se eligen en validación.**

**Regresión logística** 20 %

Relaciones simples e interpretables

**Random Forest** 30 %

Interacciones y no linealidades

**XGBoost** 50 %

Corrige errores y captura patrones

más finos

### ¿Cómo elegimos los pesos?

Probamos distintas combinaciones y comparamos su AUC en validación.

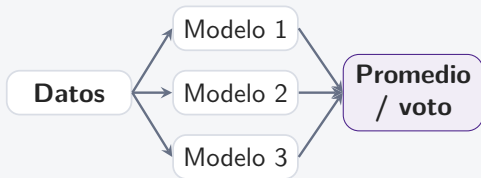
| Pesos               | AUC val.     |
|---------------------|--------------|
| 33 / 33 / 33        | 0.842        |
| <b>20 / 30 / 50</b> | <b>0.856</b> |
| 10 / 40 / 50        | 0.851        |

$$\hat{p} = 0.20\hat{p}_A + 0.30\hat{p}_B + 0.50\hat{p}_C$$

**Mayor AUC**  $\Rightarrow$  elegimos **20 / 30 / 50**.

## Bagging

Modelos en paralelo



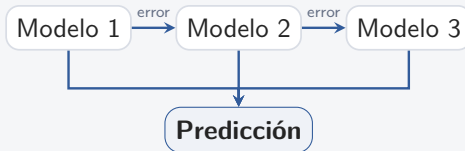
**Diversifica** para reducir varianza

Random Forest

**Bagging diversifica**

## Boosting

Modelos secuenciales



**Corrige** los errores anteriores

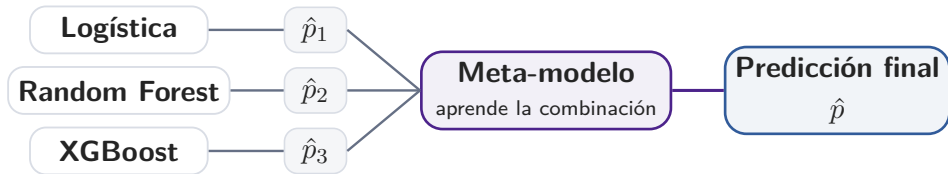
XGBoost · LightGBM · CatBoost

**Boosting corrige**

## Stacking: aprender la combinación

El **meta-modelo** aprende cómo combinar las predicciones de los modelos base.

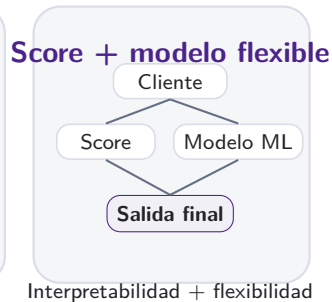
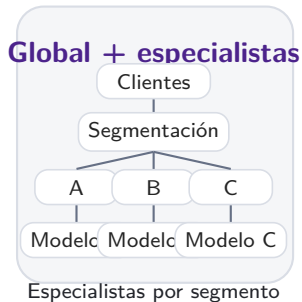
MODELOS BASE



**Stacking no promedia:** aprende cuándo confiar más en cada modelo.

**Importante:** el meta-modelo debe entrenarse con predicciones **out-of-fold** para evitar leakage.

## Modelos híbridos: una arquitectura diferente



## ¿Cuándo tiene sentido un modelo híbrido?

### 1. Patrones distintos entre grupos

La relación entre  $X$  e  $Y$  no es igual para todas las poblaciones.  
Segmentación → especialistas

### 2. Hay reglas conocidas

Restricciones, políticas o casos que no necesitan ser aprendidos.  
Reglas + ML

### 3. Existen objetivos que compiten

Por ejemplo: interpretabilidad vs. capacidad predictiva.  
Score + modelo flexible

### 4. Un modelo no captura todo

Distintas partes del problema requieren mecanismos diferentes.  
Arquitectura especializada

## Hora de practicar

Ensembles • Stacking • Modelos híbridos

[Abrir notebook en Google Colab](#)